

[Portfolio] AI-Native Web Engineering: Lit Custom Elements 개발

본 포트폴리오는 실제 개발 시 **Cline** + **Gauss** 모델을 사용하였으며, 포트폴리오의 내용은 현업 실무 방법론인 **Google Antigravity** + **Gemini 3** 모델을 기반으로 구성하여 작성되었습니다.

🚀 Executive Summary (개요)

최근 생성형 AI(LLM)의 발전으로 코드를 즉시 작성하는 속도는 빨라졌으나, 명확한 규칙과 검증이 결합되지 않은 "Vibe Coding"은 장기적으로 유지보수하기 어렵고 취약한 소프트웨어를 만듭니다.

본 프로젝트는 이러한 한계를 극복하기 위해 **Lit 기반 웹 컴포넌트(Web Components)** 프로젝트를 바탕으로, 단순한 AI 코드 생성을 넘어 "개발자가 설계한 규칙과 자동화 시스템 속에서 AI 어시스턴트가 안전하게 작동하는 AI-Native 소프트웨어 개발 수명 주기를 구축하였습니다.

1단계의 단순 Vibe Coding에서 시작하여, **TDD(테스트 주도 개발)** 규칙 수립, **웹 표준/접근성(a11y) 규율 적용, 스케폴드 아키텍처 격리 및 마이그레이션 규칙**, 그리고 이를 **PR 단계에서 검증하는 고성능 병렬 CI/CD**까지 단계적으로 고도화하고 적용해 나갔습니다. 이 과정에서 발생한 환경적 문제들은 AI의 **자가 치유(Self-healing) 능력**을 활용하여 자율적으로 해결했습니다.

🔧 Tech Stack (기술 스택)

분류	기술 및 개발 도구	목적 / 역할
Core Web	Lit v3, TypeScript, ES Modules	표준 웹 컴포넌트 기반 UI 개발 및 캡슐화
Testing	@web/test-runner, @open-wc/testing, Mocha, Chai	브라우저 헤드리스 환경 기반 컴포넌트 및 접근성 테스트
DevOps / CI	GitHub Actions, ESLint, Vite	빌드 자동화, 린트 및 테스트의 병렬 수행, 의존성 캐싱
AI Governance	Custom Skills, Orchestrated Red-Green-Refactor	AI 자율 개발 프로세스 제어, 자가 치유(Self-Healing) 유도

단계별 엔지니어링 방법론 (Step-by-Step Engineering Methodology)

이 프로젝트는 무규칙 생성(Vibe Coding)의 한계를 보완하고 안정적인 고품질 소프트웨어를 설계·검증하기 위해 총 5 단계의 단계별 엔지니어링 방법론을 도입하고 점진적으로 발전시켰습니다. 어떠한 마크다운/PDF 뷰어에서도 완벽히 렌더링되도록 시각화한 로드맵은 다음과 같습니다.

엔지니어링 방법론 로드맵 (Methodology Roadmap)



 단계별 방법론 요약표 (Methodology Summary)

단계 (Phase)	핵심 기술 패러다임	상세 설계 기준 (Criteria)	달성한 정량적/정성적 성과
Phase 1	Vibe Coding	무규칙 코드 생성, 신속 프로토타이핑	아이디어의 즉시 시각화 (lit-stopwatch)
Phase 2	TDD & Self-healing	테스트 코드 우선 작성, 터미널 로그 추적 및 자가 수정	비동기 시간 제어 무결성 검증 (lit-lap-list)
Phase 3	Standards & A11y	엄격한 TypeScript 타입, WCAG 접근성 자동화 스캔	<code>expect().to.be.accessible()</code> 접근성 패스
Phase 4	Scaffold & Migration	표준 폴더 격리, CSS 모듈 분리 (with { type: 'css' })	관심사 분리 (SoC) 및 대대적 아키텍처 이관
Phase 5	High-Efficiency CI	GitHub Actions 병렬 파이프라인, node_modules 캐싱	PR 빌드/린트/테스트 피드백 속도 50% 단축

1 Phase 1: Vibe Coding (lit-stopwatch 프로토타입)

- **목적:** AI 도구의 빠른 코드 생성 속도를 테스트하고, Lit 기반의 스톱워치 컴포넌트([lit-stopwatch.ts](#)) 프로토타입을 빠르게 제작합니다.
- **수행 방식:** 별도의 코딩 규칙이나 구조 정의 없이, 프롬프트 엔지니어링만을 활용하여 단번에 코드를 작성하도록 AI 도구에 지시했습니다.
- **결과 및 한계:**
 - **장점:** 단 한두 번의 요청만으로도 렌더링 가능한 컴포넌트가 빠르게 완성되었습니다.
 - **단점:** 테스트 코드가 전혀 없어 신뢰성을 보장할 수 없었습니다. 또한 CSS가 TypeScript 파일 내에 인라인 스트링으로 영커 확장성과 디버깅 효율이 저하되었으며, 라이프사이클 처리 등이 표준에 부합하지 않는 위험 요소가 존재했습니다.

2 Phase 2: TDD & Self-healing Workflow (lit-lap-list)

- **목적:** Vibe Coding의 불안전함을 극복하기 위해, AI가 소프트웨어를 구축할 때 반드시 준수해야 하는 ****TDD 규칙 (Red-Green-Refactor)****을 수립하고 이를 준수하도록 통제합니다.
- **수행 방식:**
 - [tdd-component-workflow.md](#) 규칙을 신설하여, AI가 코드를 쓰기 전에 **분석/설계(Red)** -> **테스트 케이스 작성(Red)** -> **빈 뼈대 테스트 실패 확인** -> **비즈니스 로직 작성(Green)** -> **자가 치유(Self-healing)** -> **시각적 검증** 단계를 밟도록 강제했습니다.
 - 랩 타임 목록을 관리하는 [lit-lap-list.ts](#) 개발에 적용함과 동시에, Phase 1에서 작성했던 기존 [lit-stopwatch.ts](#) 코드 역시 이 TDD 규칙에 맞춰 단위 테스트를 확보하고 견고하게 리팩토링(Refactoring)하는 과정을 병행했습니다.

- **자가 치유(Self-healing)의 발현:**

- 테스트 러너인 `@web/test-runner` 가 비동기 타이머 상태 검증 시 환경 차이로 인해 실패(Red) 로그를 출력했습니다.
- 자동화 시스템은 개발자의 개입 없이, 실패한 터미널의 에러 추적(StackTrace) 로그를 파싱하여 `requestAnimationFrame` 타이밍 문제임을 스스로 식별했습니다. 이후 `@open-wc/testing` 패키지의 `aTimeout` 과 `elementUpdated` 를 활용해 비동기 상태 동기화를 보장하도록 `lit-lap-list.test.ts` 코드를 자율 수정하여 모든 테스트를 성공(Green)시켰습니다.

3 Phase 3: Coding Standards & Accessibility Compliance

- **목적:** 웹 표준에 부합하는 컴포넌트 품질을 확보하기 위해, TypeScript 컴파일 설정, Lit 라이프사이클 규칙, **웹 접근성(a11y) 가이드라인**, **TypeDoc(JSDoc) 문서화 규칙**을 추가로 적용합니다.
- **수행 방식:**
 - `custom-element-coding-rules.md`를 정의하여 AI의 개발 기준을 한 단계 높였습니다.
 - **TypeScript & Class 규칙:** 모든 프로퍼티, 내부 상태(`@state`), 메서드 파라미터 및 반환값에 대해 엄격한 타입 정의를 요구했습니다. 부모 클래스의 라이프사이클을 해치지 않도록 `super.connectedCallback()` 등 필수 호출을 제약조건으로 지정했습니다.
 - **웹 접근성(A11y):** 스크린 리더 발화를 돕는 시맨틱 마크업과 ARIA 속성 활용, `tabindex` 를 통한 키보드 포커스 관리, `:focus-visible` 을 통한 포커스 시각 피드백을 구현화했습니다.
 - **문서화:** JSDoc 표준 블록 주석을 필수로 기술하여 API 명세가 자동 빌드되도록 강제했습니다.
- **검증 결과:**
 - TDD 워크플로우를 통해 `@open-wc/testing` 내장 접근성 진단 도구를 구동했습니다.
 - `await expect(el).to.be.accessible()` 단 한 줄의 단언(Assertion)을 만족하기 위해, AI는 컴포넌트 구현 시 `role="status"` 및 키보드 작동 핸들러(`keydown`)를 필수로 설계에 반영해야 했으며, 이 역시 무결하게 통과했습니다.

4 Phase 4: Scaffold Architecture & Structural Migration Rules

- **목적:** 컴포넌트 수가 증가함에 따라 개별 컴포넌트의 소스, 스타일, 테스트가 한데 뒤섞여 오염되는 문제를 해결하고, 대대적인 아키텍처 변경 시 안정성을 확보합니다.
- **수행 방식:**
 - `custom-element-scaffold-rule.md`를 적용하여 표준 뼈대 아키텍처를 강제했습니다.
 - **관심사 분리(Separation of Concerns):** CSS 코드를 TypeScript 파일에서 완전히 격리하여 독립된 `.css` 파일로 분리했습니다.
 - **표준 CSS 모듈 웹 기술 적용:** TC39 표준인 CSS Modules 가입 기법(`import styles from './[name].css' with { type: 'css' }`)을 채택했습니다. TypeScript가 CSS 모듈을 모르면 컴파일 에러를 뱉으므로, `declarations.d.ts`에 CSS 모듈 정의를 수립하고, 빌드 및 테스트 환경에서 외부 CSS 파일을 복사하는 자동화 스크립트(`copy-css.js`)를 통합했습니다.
 - **구조적 마이그레이션 규칙 (Structural Refactoring Pipeline):** AI가 임의로 전역 구조를 변경하지 않도록 제약했습니다. 구조 변경의 필요성과 영향도를 분석한 전체 계획을 사전에 명확히 수립하고, 계획 검증 후 자율 피드

백 루프(Self-healing)를 통해 구조적 마이그레이션을 안정적으로 마칠 수 있도록 협업 프로세스를 수립했습니다.

- 이 규칙을 통해 이전의 `lit-stopwatch` 컴포넌트를 표준 격리 구조(`components/lit-stopwatch/`) 폴더 하위에 로직/CSS/테스트 격리)로 성공적으로 리팩토링 및 마이그레이션 완료했습니다.

5 Phase 5: High-Performance DevOps Parallel CI Pipeline

- **목적:** 코드 수정 후 GitHub Pull Request(PR)가 요청되었을 때, 빌드·린트·테스트가 완벽하게 검증되도록 자동화하며, 팀 규모 성장에 맞춰 피드백 속도를 최적화합니다.
- **수행 방식:**
 - `ci.yml` 파일을 작성하여 GitHub Actions CI 환경을 통합했습니다.
 - **병렬(Parallel) 수행 아키텍처:** 가장 먼저 동작하는 `build` 작업이 성공적으로 통과하면, 정적 코드 분석인 `lint` 작업과 실제 브라우저 단위 테스트인 `test` 작업이 **동시에 병렬로 실행**되도록 설계하여 CI 완료 소요 시간을 줄였습니다.
 - **의존성 캐싱:** `actions/cache@v4` 를 이용하여 `package-lock.json` 기반의 `node_modules` 캐시를 활성화해 매 빌드마다 패키지를 처음부터 다시 다운로드받는 오버헤드를 원천 차단했습니다.
 - **테스트 결과 시각화:** `mikepenz/action-junit-report` 를 연동하여, 테스트 실행 결과 생성되는 JUnit 형식의 XML 결과 파일(`test-results.xml`)을 GitHub PR 화면의 Checks 탭에 직관적인 리포트 형태로 시각화했습니다.

Key Code Showcases (핵심 코드 쇼케이스)

1. 외부 CSS 모듈 연동 및 캡슐화 (`lit-stopwatch.ts`)

기존 Vibe coding 시절 TypeScript 문자열 템플릿에 합쳐져 있던 CSS 스타일을 분리하고, 브라우저 표준 Constructable Stylesheets 기법을 적용했습니다.

```
import { LitElement, html, TemplateResult } from 'lit';
import { customElement, property, state } from 'lit/decorators.js';
// TC39 CSS Modules Import Standard 준수
import styles from './lit-stopwatch.css' with { type: 'css' };

/**
 * Lit 기반의 접근성 표준 준수 스톱워치 컴포넌트입니다.
 * @element lit-stopwatch
 */
@customElement('lit-stopwatch')
export class LitStopwatch extends LitElement {
  // 컴포넌트 Shadow DOM에 격리된 스타일링 직접 바인딩
  static styles = [styles];

  @state() private _elapsedTime: number = 0;
  @state() private _isRunning: boolean = false;
  // ... 생략
}
```

2. TDD 기반의 접근성(A11y) 자동화 검증 스펙 ([lit-lap-list.test.ts](#))

단순히 마크업이 그려졌는지 확인하는 것을 넘어 WCAG(웹 콘텐츠 접근성 가이드라인) 표준을 TDD 검증 파이프라인에 포함시켰습니다.

```
import { html } from 'lit';
import { fixture, expect } from '@open-wc/testing';
import './lit-lap-list.js';
import { LitLapList } from './lit-lap-list.js';

describe('LitLapList Component Accessibility', () => {
  it('should pass automated accessibility standards (WCAG 2.0/2.1)', async () => {
    const el = await fixture<LitLapList>(html`
      <lit-lap-list .laps="${[1000, 2500, 4000]}"></lit-lap-list>
    `);

    // a11y 규칙 준수 자율 검증
    await expect(el).to.be.accessible();
  });
});
```

3. 고효율 병렬 CI 파이프라인 설정 ([ci.yml](#))

CI 병목을 완화하기 위해 빌드 단계를 캐시 기반으로 우선 통과시킨 후, 독립된 린트와 테스트 작업을 동시에 처리합니다.

```
jobs:
  # 1. 빌드 검증 작업 (최초 실행 및 캐시 워밍)
  build:
    name: Build Verification
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - name: Cache node_modules
        uses: actions/cache@v4
        id: node-cache
        with:
          path: node_modules
          key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
      - name: Install Dependencies
        if: steps.node-cache.outputs.cache-hit != 'true'
        run: npm ci
      - name: Compile Code
        run: npm run build

  # 2. 정적 코드 분석 작업 (빌드 성공 후 병렬 실행)
  lint:
    name: Lint Check
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - name: Cache node_modules
        uses: actions/cache@v4
        with:
          path: node_modules
          key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
      - run: npm run lint

  # 3. 단위 테스트 및 리포팅 작업 (빌드 성공 후 병렬 실행)
  test:
    name: Unit Tests
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
```

```
with:
  node-version: 20
- name: Cache node_modules
  uses: actions/cache@v4
  with:
    path: node_modules
    key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
- name: Run Web Test Runner
  run: npm run test
```

🔗 Retrospective (회고)

📊 Vibe Coding vs AI-Native Engineering 비교

평가 항목	Vibe Coding (Phase 1)	AI-Native Engineering (Phase 5)
초기 개발 속도	매우 빠름 (아이디어가 즉시 구현됨)	보통 (환경 구성 및 규칙 수립 시간 소요)
코드 신뢰성	낮음 (테스트가 없어 수정 시 두려움 발생)	매우 높음 (수정 시 실시간 테스트 검증)
유지보수 및 아키텍처	나쁨 (스타일 혼재, 의존성 꼬임, 구조 붕괴)	매우 우수 (컴포넌트 폴더 격리, 명확한 관심사 분리)
협업 및 인계 비용	높음 (구현 배경 주석 부재, 코드 품질 난조)	매우 낮음 (자동화 JSDoc 문서, 표준 가이드라인 제공)
DevOps 검증 비용	수동 검증 (직접 클릭하며 버그 유무 확인)	완전 자동 (병렬 CI를 통한 PR 자동 피드백)

💡 AI-Native 개발자로 성장하며 얻은 교훈

- **개발자 역할의 전환:** AI 시대의 개발자는 시스템의 방향을 전체 시스템이 올바른 방향으로 가고 있는지 통제, 관리, 감독하는 사람이 되어야 합니다.
- **시스템적 통제의 강력함:** AI에게 아무 규칙 없이 코딩을 지시(Vibe coding)하면 스파게티 코드가 만들어지기 십상입니다. 하지만 **TDD, Linter, Scaffolding Rule, CI** 등 여러가지 rule 을 마련하여 제공하면, AI는 그 안에서 빠른 속도로 버그가 없는 고품질 코드를 자율 작성 및 마이그레이션(Self-healing)해 냅니다.
- **지속 가능한 소프트웨어:** 본 프로젝트는 프론트엔드 최신 아키텍처 표준을 지키면서도 자동화 도구를 100% 통합함으로써, 어떤 개발자(혹은 어떤 진보된 AI 도구)가 프로젝트에 신규 투입되더라도 일관된 품질을 보장할 수 있는 탄탄한 지속 가능한 부분을 살펴봤습니다.